Duale Hochschule Baden-Württemberg Mannheim
Studiengang Informatik

Studienarbeit

Konzeption, Implementierung und Evaluation eines offlinefähigen Reise- und Sprachlern-Dashboards auf Basis eines Raspberry Pi

Verfasser: Michael Kurz (Matrikelnr. 6722742)
Schayan Goudarzi (Matrikelnr. 3174980)

Kurs: TINF24_IT1
Betreuer: Jürgen Schultheis
Bearbeitungszeitraum: Oktober 2026 – April 2027

3. November 2026

Verantwortlichkeiten

Diese Arbeit ist eine Gemeinschaftsarbeit. Die folgende Übersicht zeigt die Hauptverantwortung je Bereich; innerhalb der Kapitel ist der jeweils verantwortliche Verfasser zusätzlich in Klammern gekennzeichnet.

Bereich	Verantwortlich
Systemarchitektur, Backend, Geschäftslogik, Security	Michael Kurz
Frontend, Datenbankanbindung, Raspberry Pi, Deployment	Schayan Goudarzi
Konzept, Marktanalyse, Umfrage, Evaluation, Dokumentation	gemeinsam

Eigenständigkeitserklärung

Gemäß § 5 (3) der „Studien- und Prüfungsordnung DHBW-Technik“

Hiermit versichern wir, Schayan Goudarzi und Michael Kurz, durch unsere Unterschriften, die vorliegende Studienarbeit ohne Hilfe Dritter und nur mit den angegebenen Quellen und Hilfsmitteln angefertigt zu haben. Keine weiteren Personen waren an der geistigen Herstellung beteiligt. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder unveröffentlichten Schriften oder dem Internet entnommen worden sind, sind als solche kenntlich gemacht. Wir versichern zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

Zur sprachlichen Überarbeitung und als Formulierungshilfe wurden KI-basierte Werkzeuge eingesetzt. Die inhaltliche Erarbeitung sowie die Verantwortung für den Inhalt der Arbeit liegen vollständig bei den Verfassern.

Mannheim, 3. November 2026

Michael Kurz	Schayan Goudarzi

Kurzfassung

Hier kommt die Kurzfassung (Abstract): Problem, Ziel, Vorgehen und wichtigstes Ergebnis in wenigen Sätzen.

Inhaltsverzeichnis

Verantwortlichkeiten 2

Eigenständigkeitserklärung 3

Kurzfassung 4

1 Einleitung 7

1.1 Einordnung und Motivation . . . 7
1.2 Problemstellung . . . 8
1.3 Zielsetzung . . . 8
1.4 Aufgabenstellung . . . 8
1.5 Aufbau der Arbeit . . . 9

2 Grundlagen 10

2.1 Begriffliche Grundlagen . . . 10
2.2 Usability und Anforderungsermittlung . . . 11
2.3 Offlinefähige Webanwendungen . . . 13

3 Marktanalyse 15

3.1 Vergleichbare Systeme . . . 15
3.2 Bewertung nach Kriterien . . . 16
3.3 Abgeleitete Lücke . . . 17

4 Anforderungsanalyse 18

4.1 Umfrage und Datenerhebung . . . 18
4.2 Funktionale Anforderungen . . . 18
4.3 Nichtfunktionale Anforderungen . . . 18
4.4 Priorisierung (MoSCoW) . . . 18

5 Konzeption 19

5.1 Systemarchitektur . . . 19
5.2 Datenmodell . . . 19
5.3 Oberfläche und Wireframes . . . 19

6 Implementierung **20**
6.1 Backend 20
6.2 Frontend 20
6.3 Datenhaltung 20
6.4 Betrieb auf dem Raspberry Pi 20

7 Evaluation **21**
7.1 Verifikation (Funktions- und Datenbanktests) 21
7.2 Validierung (Usability-Test) 21

8 Kritische Reflexion **22**

9 Ausblick **23**

10 Methoden und Verfahren **24**
10.1 Vorgehen 24
10.2 Eingesetzte Werkzeuge und Versionen 24

Literatur **25**

1 Einleitung

1.1 Einordnung und Motivation

Im Jahr 2025 unternahmen 80,5 % der deutschsprachigen Bevölkerung ab 14 Jahren mindestens eine Urlaubsreise von fünf oder mehr Tagen, und 78 % dieser Reisen führten ins Ausland [1]. Vor Reiseantritt ist einiges vorzubereiten. Dazu gehören organisatorische Aufgaben wie das Planen der einzelnen Reisetage, das Erstellen einer Packliste und die Übersicht über das Budget, aber auch das Erlernen grundlegender Wörter und Redewendungen in der Sprache des Ziellandes.

Beide Bereiche hängen zusammen: Der Wunsch, ins Ausland zu reisen, zählt zu den stärksten praktischen Motiven für das Erlernen einer Fremdsprache, wie eine Befragung von 229 Studierenden zeigt [2], und umgekehrt erleichtern Sprachkenntnisse die Orientierung vor Ort. In der Praxis werden Reiseorganisation und Sprachenlernen dennoch mit getrennten Werkzeugen bearbeitet.

Für beide Bereiche existieren zahlreiche digitale Anwendungen; allein für die Urlaubsplanung nutzen Reisende im Schnitt vier verschiedene Apps [3]. Die Reiseplanung decken Dienste wie Wanderlog [4] oder TripIt [5] ab, für das Sprachenlernen stehen Applikationen wie Duolingo [6], Babbel [7] oder Anki [8] zur Verfügung. Diese Produkte konzentrieren sich jeweils nur auf einen Teilbereich, setzen in der Regel eine kontinuierliche Internetverbindung voraus und speichern Inhalte sowie persönliche Nutzerdaten in der Cloud, wie die Marktanalyse in Kapitel 3 zeigt. Wer eine Reise vorbereitet, muss seine Daten daher über mehrere Anbieter verteilen und gibt die Kontrolle darüber teilweise ab.

Gerade im Ausland ist eine stabile Internetverbindung aber nicht immer gegeben. Die Inlandspreise des EU-Roamings gelten nur innerhalb der EU; außerhalb dürfen die Anbieter weiterhin Aufschläge erheben, sodass insbesondere die mobile Datennutzung dort meist wesentlich teurer ist [9], [10]. Das betrifft alle drei in dieser Arbeit betrachteten Regionen. Unterwegs, wo die Apps am ehesten gebraucht werden, sind cloudgebundene Anwendungen damit nur eingeschränkt nutzbar.

Beide Bereiche werden somit bislang getrennt und überwiegend cloudgebunden abgedeckt. Daraus ergibt sich die im folgenden Abschnitt dargestellte Problemstellung.

1.2 Problemstellung

Die Vorbereitung einer Auslandsreise umfasst zwei Aufgabenbereiche, die bislang getrennt voneinander bearbeitet werden: a) die organisatorische Planung der Reise und b) die sprachliche Vorbereitung auf das jeweilige Zielland. Für beide Bereiche existieren zwar etablierte Anwendungen, eine integrierte Lösung, die beide Bereiche verbindet und dabei ohne ständige Internetverbindung sowie ohne Auslagerung der persönlichen Daten an externe Anbieter auskommt, fehlt bislang jedoch.

Diese Studienarbeit untersucht daher, inwiefern sich ein offlinefähiges, lokal auf einem Raspberry Pi betriebenes Dashboard umsetzen lässt, das Reiseplanung und Sprachenlernen in einer einzigen Anwendung vereint. Diese Leitfrage wird in drei Teilfragen untergliedert. Zunächst ist zu klären, welche funktionalen und nichtfunktionalen Anforderungen potenzielle Nutzer an ein solches System stellen. Darauf aufbauend stellt sich die Frage, wie sich ein solches System technisch auf einem Raspberry Pi umsetzen lässt. Abschließend ist zu untersuchen, inwieweit der entwickelte Prototyp die ermittelten Anforderungen hinsichtlich Funktionalität und Benutzbarkeit erfüllt.

1.3 Zielsetzung

Die Arbeit zielt nicht darauf, bestehende Reiseplaner oder Sprachlern-Apps zu ersetzen, sondern beide Funktionen in einer lokal betriebenen, offlinefähigen Anwendung zusammenzuführen. Entwickelt wird ein Prototyp eines Reise- und Sprachlern-Dashboards auf Basis eines Raspberry Pi, der ohne Internetverbindung im lokalen Netzwerk läuft. Im Einzelnen soll die Anwendung die Planung einer Reise an einem Ort bündeln, also Reiseziele, Tagesplanung, Packlisten und Budget, und zugleich das Lernen der Zielsprache durch Vokabeln, Lektionen und Quiz unterstützen. Der Lernfortschritt wird dabei je Benutzer nachvollziehbar festgehalten. Die gesamte Anwendung soll vollständig offline und datensparsam auf lokaler Hardware funktionieren. Betrachtet werden exemplarisch die Regionen Russland, Japan und Korea. Als erfolgreich gilt die Umsetzung, wenn die aus der Problemstellung abgeleiteten Aufgaben nachvollziehbar bearbeitet und die zuvor erhobenen Anforderungen durch den Prototyp erfüllt werden.

1.4 Aufgabenstellung

Aus der zuvor beschriebenen Problemstellung ergeben sich die folgenden Aufgaben, die im weiteren Verlauf der Arbeit bearbeitet werden:

1. Grundlagen und Marktanalyse
   1.1. Vergleichbare Systeme analysieren (Reiseplaner, Sprachlern-Apps, lokale Lösungen)

   1.2. Bewertungskriterien aus der Literatur ableiten
   1.3. Die Marktlücke bestimmen
2. Anforderungsanalyse
   2.1. Nutzeranforderungen erheben (Umfrage)
   2.2. Funktionale und nichtfunktionale Anforderungen ableiten
   2.3. Anforderungen priorisieren (MoSCoW)
3. Konzeption
   3.1. Systemarchitektur entwerfen
   3.2. Datenmodell entwerfen
   3.3. Oberfläche konzipieren (Wireframes)
4. Implementierung
   4.1. Prototyp auf Basis von Spring Boot umsetzen
   4.2. Offline-Betrieb auf dem Raspberry Pi einrichten
5. Evaluation
   5.1. Verifikation durch Funktions- und Datenbanktests
   5.2. Validierung durch einen Usability-Test mit Testnutzern

1.5 Aufbau der Arbeit

Die Arbeit gliedert sich in zehn Kapitel. Kapitel 2 stellt die Grundlagen bereit. Kapitel 3 analysiert vergleichbare Systeme und leitet die Marktlücke ab, Kapitel 4 erhebt und priorisiert die Anforderungen. Kapitel 5 entwickelt das Konzept aus Architektur, Datenmodell und Oberfläche, Kapitel 6 schildert die Implementierung und den Betrieb auf dem Raspberry Pi. Kapitel 7 evaluiert das Ergebnis, Kapitel 8 reflektiert es kritisch und Kapitel 9 gibt einen Ausblick. Kapitel 10 fasst die eingesetzten Methoden und Werkzeuge zusammen.

2 Grundlagen

Dieses Kapitel stellt die Begriffe und Konzepte bereit, auf denen die folgenden Kapitel aufbauen. Die Auswahl beschränkt sich auf jene Konzepte, die für das Verständnis der Marktanalyse, der Anforderungsanalyse und der Evaluation notwendig sind: die zentralen Begriffe der Arbeit und der eingesetzte Technologiestapel (Abschnitt 2.1), die Normen und Methoden zu Softwarequalität, Gebrauchstauglichkeit und Anforderungsermittlung (Abschnitt 2.2) sowie die technischen Ansätze für offlinefähige Webanwendungen (Abschnitt 2.3).

2.1 Begriffliche Grundlagen

Few beschreibt ein *Dashboard* als visuelle Darstellung der wichtigsten Informationen, die zum Erreichen eines oder mehrerer Ziele benötigt werden, so verdichtet und auf einem Bildschirm angeordnet, dass sie auf einen Blick erfasst werden können [11]. Diese Definition stammt aus der Überwachung von Kennzahlen. In dieser Arbeit wird der Begriff etwas weiter gefasst: Das Reise- und Sprachlern-Dashboard bündelt auf einer gemeinsamen Startseite nicht nur Informationen, etwa die nächste Reise und die heute fälligen Vokabeln, sondern auch den Einstieg in die Funktionen beider Bereiche.

Eine Anwendung gilt in dieser Arbeit als *offlinefähig*, wenn sämtliche Funktionen ohne Verbindung zum Internet nutzbar sind. Das schließt nicht aus, dass die Anwendung über ein lokales Netzwerk erreicht wird; entscheidend ist, dass keine Komponente auf Dienste außerhalb dieses Netzwerks angewiesen ist. ISO/IEC 25010 nennt den Betrieb ohne Netzverbindung ausdrücklich als Nutzungskontext, auf den ein Produkt ausgelegt sein kann [12, Abschn. 3.8]. Davon zu unterscheiden ist die *lokale Datenhaltung*: Die Daten liegen auf einem Gerät unter Kontrolle der Nutzer und nicht auf den Servern eines Anbieters. Beide Eigenschaften treten getrennt voneinander auf, wie die Marktanalyse in Kapitel 3 zeigt; der Prototyp dieser Arbeit soll beide erfüllen.

Der Sprachlernbereich stützt sich auf das Prinzip der *verteilten Wiederholung* (Spaced Repetition). Ebbinghaus zeigte 1885 in Selbstversuchen mit sinnlosen Silben, dass das Vergessen unmittelbar nach dem Lernen am stärksten ist und sich danach verlangsamt, und dass über mehrere Tage verteilte Wiederholungen wirksamer sind als dieselbe Zahl von Wiederholungen an einem Tag [13]. Leitner übertrug dieses Ergebnis in ein praktisches Verfahren: Karteikarten wandern durch mehrere Fächer, eine richtig beantwortete Karte

rückt ein Fach weiter und wird seltener abgefragt, eine falsch beantwortete fällt in das erste Fach zurück [14]. Anwendungen wie Anki setzen dieses Prinzip um (Kapitel 3); der Prototyp verwendet eine Variante mit fünf Fächern, die in Kapitel 6 beschrieben wird.

Als Hardware dient ein *Raspberry Pi 5*, ein Einplatinencomputer mit einem 64-Bit-Vierkernprozessor (Arm Cortex-A76, 2,4 GHz), wahlweise 1 bis 16 GB Arbeitsspeicher, Gigabit-Ethernet und WLAN nach 802.11ac, auf dem das Debian-basierte Raspberry Pi OS läuft [15]. Für diese Arbeit wird die Variante mit 4 GB eingesetzt. Die Anwendung selbst wird mit *Spring Boot* entwickelt, einem Java-Framework, das eine Webanwendung samt eingebettetem Webserver als einzelne ausführbare Datei verpackt und durch Voreinstellungen den Konfigurationsaufwand gering hält [16]. Die Oberfläche wird mit *Thymeleaf* erzeugt, einer Template-Engine, die HTML-Seiten auf dem Server mit Daten füllt, sodass der Browser fertige Seiten erhält und keine clientseitige Anwendungslogik benötigt [17]. Die Daten hält die relationale Datenbank *H2*, die in Java geschrieben ist und im eingebetteten Modus innerhalb der Anwendung läuft und ihre Daten in einer lokalen Datei ablegt [18]. Dieser Stapel wird in Kapitel 5 begründet und in Kapitel 10 mit Versionsnummern dokumentiert.

Die Normen und Methoden, mit denen Qualität und Anforderungen in dieser Arbeit bewertet und erhoben werden, folgen im nächsten Abschnitt.

2.2 Usability und Anforderungsermittlung

Die Norm ISO/IEC 25010 definiert ein Qualitätsmodell für Softwareprodukte, das die Produktqualität in neun Merkmale gliedert: funktionale Eignung, Leistungseffizienz, Kompatibilität, Interaktionsfähigkeit, Zuverlässigkeit, Sicherheit, Wartbarkeit, Flexibilität und Betriebssicherheit; jedes Merkmal ist in Teilmerkmale unterteilt [12, Abschn. 4.1]. Für diese Arbeit sind vier Merkmale relevant. Die *funktionale Eignung* beschreibt, ob ein Produkt Funktionen bereitstellt, die den Bedürfnissen der vorgesehenen Nutzer entsprechen, mit den Teilmerkmalen Vollständigkeit, Korrektheit und Angemessenheit. Die *Zuverlässigkeit* umfasst unter anderem die Verfügbarkeit, also die Fähigkeit, betriebsbereit und erreichbar zu sein, wenn das Produkt benötigt wird. Zur *Sicherheit* gehört die Vertraulichkeit, also dass Daten nur den dazu Berechtigten zugänglich sind; die *Interaktionsfähigkeit*, in der Vorgängerfassung von 2011 noch als Usability bezeichnet, fasst Teilmerkmale wie Erlernbarkeit, Bedienbarkeit und Schutz vor Benutzerfehlern zusammen [12, Abschn. 3]. Die Norm betont, dass die Gewichtung der Merkmale vom Nutzungskontext abhängt, also von Nutzern, Zielen, Umgebung und technischem Kontext [12, Abschn. 4.2]. Das Modell dient in Kapitel 3 als Grundlage der Vergleichskriterien und in Kapitel 4 zur Einordnung der nichtfunktionalen Anforderungen.

Die *Gebrauchstauglichkeit* (Usability) wird in DIN EN ISO 9241-11 definiert als das Ausmaß, in dem ein System durch bestimmte Benutzer in einem bestimmten Nutzungs-

kontext genutzt werden kann, um bestimmte Ziele effektiv, effizient und zufriedenstellend zu erreichen [19, Abschn. 3.1.1]. Die drei Komponenten sind dort einzeln definiert: *Effektivität* als Genauigkeit und Vollständigkeit, mit denen Benutzer ihre Ziele erreichen, *Effizienz* als Verhältnis der eingesetzten Ressourcen, etwa Zeit und Aufwand, zu den erreichten Ergebnissen, und *Zufriedenstellung* als Ausmaß, in dem die Reaktionen der Benutzer ihren Erfordernissen und Erwartungen entsprechen [19, Abschn. 3.1.12 bis 3.1.14]. Der *Nutzungskontext* ist die Kombination aus Benutzern, Zielen, Aufgaben, Ressourcen und Umgebung [19, Abschn. 3.1.15]. Während ISO/IEC 25010 damit Eigenschaften des Produkts beschreibt, bezieht sich die Gebrauchstauglichkeit auf die tatsächliche Nutzung und lässt sich nur mit Nutzern messen. Die Validierung in Kapitel 7 folgt dieser Definition: Testnutzer bearbeiten Aufgaben, wobei Effektivität über den Aufgabenerfolg, Effizienz über die benötigte Zeit und Zufriedenstellung über einen Fragebogen erfasst werden.

Ergänzend zu den Normen werden die zehn Usability-Heuristiken von Nielsen herangezogen, ein Satz von Faustregeln für die Gestaltung von Benutzeroberflächen, darunter die Sichtbarkeit des Systemstatus, die Übereinstimmung zwischen System und Lebenswelt der Nutzer, Konsistenz, Fehlervermeidung und die Bevorzugung von Wiedererkennen gegenüber Erinnern [20]. Sie sind keine Norm, aber in der Praxis weit verbreitet und dienen in Kapitel 5 als Prüfliste für den Entwurf der Oberfläche.

Das *Requirements Engineering* umfasst nach Herrmann das Ermitteln, das Dokumentieren, das Prüfen und Abstimmen sowie das Verwalten von Anforderungen [21, Abschn. 1.3]. Bei den Anforderungen an das Produkt werden *funktionale Anforderungen*, die festlegen, was das System tun soll, von *nichtfunktionalen Anforderungen* unterschieden, die das Wie beschreiben, etwa wie schnell oder wie sicher; die nichtfunktionalen Anforderungen gliedern sich weiter in Qualitätsanforderungen und Randbedingungen [21, Abschn. 5.4]. Für die Qualitätsanforderungen empfiehlt Herrmann, die Qualitätsmerkmale der ISO/IEC 25010 als Ausgangspunkt zu nehmen und sich auf die kritischen zu beschränken [21, Abschn. 5.7.4]; so wird in Kapitel 4 verfahren. Als Erhebungstechnik kommt in dieser Arbeit eine quantitative Online-Befragung mit geschlossenen Fragen und Likert-Skalen zum Einsatz. Die Teilnehmenden wurden nicht zufällig, sondern über persönliche Kontakte und Umfrage-Tauschplattformen gewonnen. Es handelt sich damit um eine Gelegenheitsstichprobe, also eine willkürliche Auswahl von Personen, die gerade zur Verfügung stehen oder leicht zugänglich sind, ergänzt um das Schneeballverfahren, bei dem Teilnehmende über ihre persönlichen Netzwerke weitere Personen anwerben. Beide Verfahren sind nach Döring und Bortz nicht-probabilistisch und können keine Repräsentativität beanspruchen; Rückschlüsse sind nur auf die Gesamtheit möglich, aus der die Stichprobe tatsächlich stammt [22, Abschn. 9.3.1]. Diese Einschränkung wird in Kapitel 4 bei der Auswertung berücksichtigt. Zur Priorisierung der ermittelten Anforderungen dient die MoSCoW-Methode, die jede Anforderung einer von vier Klassen zuordnet: Must (zwingend), Should (wichtig, aber nicht zwingend), Could (wünschenswert) und Won't

(bewusst ausgeschlossen) [23]. Sie wurde für die schnelle Anwendungsentwicklung entworfen und eignet sich für einen Prototyp mit begrenzter Entwicklungszeit, weil sie den verbindlichen Kern von den Erweiterungen trennt.

Offen ist noch, auf welchem technischen Weg eine Webanwendung ohne Internetverbindung betrieben werden kann.

2.3 Offlinefähige Webanwendungen

Eine klassische Webanwendung besteht aus einem Server, der die Anwendungslogik ausführt und Daten speichert, und einem Browser, der Seiten vom Server anfordert und darstellt. Fällt die Verbindung zwischen beiden aus, ist die Anwendung nicht mehr nutzbar. Um eine Webanwendung offlinefähig zu machen, gibt es zwei grundsätzliche Ansätze, die sich darin unterscheiden, wo Logik und Daten liegen.

Der erste Ansatz verlagert Logik und Daten in den Browser. Progressive Web Apps (PWA) nutzen dafür einen Service Worker, ein Skript, das im Hintergrund des Browsers läuft, Anfragen abfängt und zuvor zwischengespeicherte Seiten, Skripte und Daten auch ohne Netzverbindung ausliefert; Änderungen werden lokal gespeichert und bei der nächsten Verbindung mit dem Server abgeglichen [24], [25]. Dieser Ansatz macht die Anwendung auf dem jeweiligen Endgerät unabhängig vom Netz, hat aber Folgen: Die Anwendungslogik muss in JavaScript im Browser laufen, der Offline-Stand gilt nur für das Gerät, auf dem er zwischengespeichert wurde, und die eigentliche Datenhaltung bleibt in der Regel beim Server des Anbieters. Die in Kapitel 3 betrachteten kommerziellen Apps arbeiten, auch wenn sie als native Apps und nicht als PWA umgesetzt sind, nach demselben Muster: Inhalte werden auf das Gerät geladen, die Daten liegen beim Anbieter.

Der zweite Ansatz lässt die klassische Architektur unverändert und verlagert stattdessen den Server in das lokale Netzwerk der Nutzer. Die Anwendung läuft auf einem eigenen Gerät, hier dem Raspberry Pi, und alle Endgeräte im selben WLAN erreichen sie über den Browser. Eine Internetverbindung wird an keiner Stelle benötigt, die Daten verbleiben auf dem Pi, und mehrere Benutzer teilen sich dieselbe Anwendung und denselben Datenbestand. Für die Umsetzung genügt serverseitiges Rendering mit Thymeleaf; der Browser bleibt ein reiner Anzeigeclient. Der Preis dafür ist, dass die Anwendung nur dort erreichbar ist, wo der Pi erreichbar ist: zu Hause bei der Reisevorbereitung ohne Einschränkung, unterwegs nur, wenn der Pi mitgeführt und mit einem lokalen Netz, etwa dem Hotspot eines Smartphones, verbunden wird.

Diese Arbeit wählt den zweiten Ansatz. Ausschlaggebend sind die Datenhoheit der Nutzer und die gemeinsame Nutzung durch mehrere Profile auf einem Gerät. Dazu kommt, dass der Client einfach bleibt und der Prototyp mit dem in Abschnitt 2.1 beschriebenen Stapel umsetzbar ist. Die Einschränkung der Erreichbarkeit wird in Kapitel 8 kritisch reflektiert. Kapitel 3 untersucht als Nächstes, welche vergleichbaren Systeme am Markt

existieren und welche Lücke sie offenlassen.

3 Marktanalyse

Dieses Kapitel bearbeitet die Aufgaben 1.1 bis 1.3: Es stellt vergleichbare Systeme vor, bewertet sie anhand von Kriterien, die aus der Literatur abgeleitet sind, und bestimmt daraus die Marktlücke, die der Prototyp schließen soll. Die Analyse stützt sich auf die öffentlich zugängliche Produktdokumentation, die Preisseiten der Anbieter und die Beschreibungen in den App-Stores (Stand: 2. November 2026). Ein eigener Praxistest der Anwendungen wurde nicht durchgeführt; die Aussagen beziehen sich daher auf die von den Herstellern dokumentierten Funktionen.

3.1 Vergleichbare Systeme

Betrachtet werden kommerzielle Reiseplaner, kommerzielle Sprachlern-Apps und quelloffene Anwendungen, die auf eigener Hardware betrieben werden (self-hosted). Aus jeder Gruppe wurden verbreitete Vertreter ausgewählt, deren Funktionsumfang sich mit dem geplanten Dashboard überschneidet.

Wanderlog ist ein Reiseplaner für Web und Mobilgeräte, der Reiseziele, Tagespläne und Buchungen in einer gemeinsamen Karten- und Listenansicht zusammenführt und zusätzlich eine Budgetverfolgung anbietet. Die Grundversion ist kostenlos; der Offline-Zugriff auf die Reisepläne, die Routenoptimierung und der Export nach Google Maps sind der Pro-Version vorbehalten, die für 39,99 USD pro Jahr angeboten wird [26].

TripIt ist kein Planungswerkzeug im engeren Sinn, Packlisten oder ein Budget gibt es dort nicht. Die Anwendung erzeugt aus weitergeleiteten Buchungsbestätigungen automatisch einen Reiseplan und dient damit der Organisation bereits gebuchter Reisen. Der Reiseplan ist laut Hersteller auch ohne Internetverbindung abrufbar; Zusatzfunktionen wie Flugstatusmeldungen sind Teil des Abonnements TripIt Pro, das im deutschen App Store 48,99 EUR pro Jahr kostet [27].

Für das Sprachenlernen wurden Anki und LingoDeer betrachtet. Anki ist ein quelloffenes Karteikartenprogramm, das Vokabeln nach dem Prinzip der verteilten Wiederholung (Spaced Repetition) abfragt. Die Desktop-Version ist kostenlos, die Karten werden lokal gespeichert, und der Abgleich zwischen Geräten über den Dienst AnkiWeb ist ein optionaler Zusatz [8]. Die iOS-Version AnkiMobile wird einmalig mit 29,99 EUR berechnet [28]. Vorgefertigte Lektionen enthält Anki nicht; Inhalte müssen selbst erstellt oder aus geteilten Kartensätzen importiert werden.

LingoDeer ist eine kommerzielle Sprachlern-App mit Schwerpunkt auf ostasiatischen Sprachen wie Japanisch und Koreanisch, die strukturierte Lektionen, Vokabeln und Übungen bereitstellt und über einen Offline-Modus verfügt, für den die Lektionen vorab heruntergeladen werden [29]. Kostenlos sind lediglich die ersten Lektionen; der vollständige Zugang kostet 14,99 USD pro Monat beziehungsweise 95,99 USD pro Jahr [30]. Duolingo und Babbel, die in der Einleitung genannt wurden, werden nicht gesondert betrachtet, da LingoDeer als Vertreter der abonnementbasierten Sprachlern-Apps mit seinem Sprachangebot näher am Zielsprachenumfang dieser Arbeit liegt.

Als Vertreter der self-hosted-Lösungen wurde AdventureLog aufgenommen, ein quelloffenes Reisetagebuch mit Reiseplanung und Weltkarte, das unter der GPL-3.0 steht und per Docker auf eigener Hardware betrieben wird [31]. Die Daten verbleiben damit auf dem eigenen Server. Als Webanwendung ist AdventureLog allerdings nur nutzbar, solange der Client den Server erreicht; einen Sprachlernbereich gibt es nicht.

Für den Vergleich fehlen noch die Kriterien.

3.2 Bewertung nach Kriterien

Die Kriterien orientieren sich am Qualitätsmodell der Norm ISO/IEC 25010, die die Qualität eines Softwareprodukts in Merkmale wie funktionale Eignung, Zuverlässigkeit und Sicherheit gliedert [12]. Daraus wurden sechs Kriterien abgeleitet, die sich anhand der Herstellerangaben prüfen lassen. Die funktionale Eignung wird über drei Kriterien erfasst: den Umfang der Reisefunktionen (Reiseziele, Tagesplanung, Packliste, Budget), den Umfang der Sprachfunktionen (Vokabeln, Lektionen, Quiz, Fortschritt) und die Frage, ob beide Bereiche in einer Anwendung kombiniert sind. Die Offlinefähigkeit beschreibt, ob die Anwendung ohne Internetverbindung nutzbar ist, und betrifft damit die Zuverlässigkeit im Sinne der Verfügbarkeit. Die Datenhaltung erfasst, ob die Nutzerdaten lokal oder beim Anbieter gespeichert werden, und bildet die Vertraulichkeit aus dem Merkmal Sicherheit ab. Die Kosten sind kein Qualitätsmerkmal der Norm, für die Zielgruppe aber relevant und werden deshalb ergänzend aufgenommen.

Die Gebrauchstauglichkeit nach ISO 9241-11, verstanden als Effektivität, Effizienz und Zufriedenheit in einem bestimmten Nutzungskontext [19], lässt sich nur empirisch mit Nutzern beurteilen und ist deshalb nicht Teil dieses Vergleichs. Sie wird in Kapitel 7 für die Validierung des Prototyps herangezogen. Tabelle 3.1 fasst die Bewertung zusammen.

Keines der betrachteten Systeme deckt beide Bereiche ab. Die Offlinefähigkeit ist bei den kommerziellen Produkten entweder an das kostenpflichtige Abonnement gebunden (Wanderlog, LingoDeer) oder auf den Abruf bereits angelegter Pläne beschränkt (TripIt). Eine lokale Datenhaltung bieten nur Anki und AdventureLog, also die beiden quelloffenen Anwendungen; die übrigen Produkte sind kontobasierte Cloud-Dienste, bei denen die Daten beim Anbieter liegen.

System	Reise	Sprache	Offline	Daten	Kosten
Wanderlog [26]	Ziele, Tagesplan, Budget	nein	nur Pro-Version	Anbieter-Cloud	gratis, Pro 39,99 USD/Jahr
TripIt [27]	Reiseplan aus Buchungen	nein	Abruf des Plans	Anbieter-Cloud	gratis, Pro 48,99 EUR/Jahr
Anki [8], [28]	nein	Vokabeln (Spaced Repetition)	vollständig	lokal, Sync optional	Desktop gratis, iOS 29,99 EUR
LingoDeer [29], [30]	nein	Lektionen, Vokabeln, Übungen	Offline-Modus, Lektionen vorab laden	Anbieter-Cloud	erste Lektionen gratis, 14,99 USD/Monat
AdventureLog [31]	Ziele, Tagesplan, Tagebuch	nein	nur im eigenen Netz	eigener Server	gratis (GPL-3.0), eigene Hardware
Prototyp dieser Arbeit	Ziele, Tagesplan, Packliste, Budget	Vokabeln, Lektionen, Quiz, Fortschritt	vollständig	Raspberry Pi	keine laufenden Kosten

Tabelle 3.1: Kriteriengestützter Vergleich der betrachteten Systeme (Stand 2. November 2026)

3.3 Abgeleitete Lücke

Der Vergleich zeigt, dass Reiseplanung und Sprachenlernen am Markt getrennt bedient werden. Für die Reiseplanung gibt es Produkte, die aber entweder cloudgebunden sind oder, wie AdventureLog, zwar lokal betrieben werden, jedoch keinen Sprachlernbereich bieten. Für das Sprachenlernen gilt das Umgekehrte: LingoDeer liefert fertige Lerninhalte, bindet sie aber an ein Abonnement und an die Infrastruktur des Anbieters; Anki arbeitet lokal und offline, bringt aber weder Lerninhalte noch einen Reisebezug mit.

Die Lücke liegt in der Kombination: beide Bereiche in einer Anwendung, vollständige Offlinefähigkeit, Datenhaltung auf eigener Hardware und keine laufenden Kosten gibt es jeweils einzeln, aber in keinem der Systeme zusammen. Lokale Datenhaltung findet sich bei Anki und AdventureLog, der Funktionsumfang orientiert sich an Wanderlog und LingoDeer. Der Prototyp soll diese vier Eigenschaften zusammen erfüllen. Damit ist Aufgabe 1 bearbeitet. Ob potenzielle Nutzer diese Lücke ebenso sehen und welche Funktionen ihnen wichtig sind, klärt Kapitel 4.

4 Anforderungsanalyse

4.1 Umfrage und Datenerhebung

4.2 Funktionale Anforderungen

4.3 Nichtfunktionale Anforderungen

4.4 Priorisierung (MoSCoW)

5 Konzeption

5.1 Systemarchitektur

5.2 Datenmodell

5.3 Oberfläche und Wireframes

6 Implementierung

6.1 Backend

6.2 Frontend

6.3 Datenhaltung

6.4 Betrieb auf dem Raspberry Pi

7 Evaluation

7.1 Verifikation (Funktions- und Datenbanktests)

7.2 Validierung (Usability-Test)

8 Kritische Reflexion

9 Ausblick

10 Methoden und Verfahren

10.1 Vorgehen

10.2 Eingesetzte Werkzeuge und Versionen

Werkzeug / Bibliothek	Version	Zweck
Java (OpenJDK)	21	Programmiersprache
Spring Boot	3.5.16	Backend-Framework
Thymeleaf	(Spring Boot)	Server-seitiges Rendering
Bootstrap	5.3.8	Oberfläche
Bootstrap Icons	1.13.2	Icons
H2 Database	2.3.232	lokale Datenbank
Maven	3.x	Build

Literatur

[1] Forschungsgemeinschaft Urlaub und Reisen e. V. (FUR), „Reiseanalyse 2026: Erste ausgewählte Ergebnisse der 56. Reiseanalyse," Forschungsgemeinschaft Urlaub und Reisen e. V., Kiel, 2026. besucht am 2. Nov. 2026. Adresse: https://reiseanalyse.de/wp-content/uploads/2026/02/RA2026_Erste_Ergebnisse_Broschuere_DE.pdf

[2] M. D. Falcón Falcón und P. A. Arias Arroyo, „Instrumental and Integrative Motivation in University Students," *Revista Scientific*, Jg. 9, Nr. 34, S. 22–40, 2024. DOI: 10.29394/Scientific.issn.2542-2987.2024.9.34.1.22-40

[3] Bitkom e. V. „Bürger nutzen im Schnitt 4 Reise-Apps für ihre Urlaubsplanung," besucht am 2. Nov. 2026. Adresse: https://www.bitkom.org/Presse/Presseinformation/Buerger-nutzen-im-Schnitt-4-Reise-Apps-fuer-ihre-Urlaubsplanung

[4] Wanderlog. „Wanderlog: Reiseplaner," besucht am 2. Nov. 2026. Adresse: https://wanderlog.com/

[5] TripIt. „TripIt: Travel Planner," besucht am 2. Nov. 2026. Adresse: https://www.tripit.com/

[6] Duolingo. „Duolingo," besucht am 2. Nov. 2026. Adresse: https://www.duolingo.com/

[7] Babbel. „Babbel," besucht am 2. Nov. 2026. Adresse: https://www.babbel.com/

[8] Anki. „Anki: Powerful, intelligent flash cards," besucht am 2. Nov. 2026. Adresse: https://apps.ankiweb.net/

[9] Bundesnetzagentur. „Sorgenfreies Roaming in den Sommerferien," besucht am 2. Nov. 2026. Adresse: https://www.bundesnetzagentur.de/SharedDocs/Pressemitteilungen/DE/2023/20230620_Roaming.html

[10] Verbraucherzentrale NRW. „Roaming in der EU: Keine Aufschläge – mit Ausnahmen," besucht am 2. Nov. 2026. Adresse: https://www.verbraucherzentrale.nrw/roaming

[11] S. Few, *Information Dashboard Design: The Effective Visual Communication of Data*. Sebastopol: O'Reilly, 2006, ISBN: 978-0-596-10016-2.

[12] *ISO/IEC 25010:2023 Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model*, International Organization for Standardization, Genf, 2023. besucht am 2. Nov. 2026. Adresse: https://www.iso.org/standard/78176.html

[13] H. Ebbinghaus, *Über das Gedächtnis: Untersuchungen zur experimentellen Psychologie*. Leipzig: Duncker & Humblot, 1885. besucht am 3. Nov. 2026. Adresse: https://archive.org/details/b28111710

[14] S. Leitner, *So lernt man lernen: Der Weg zum Erfolg*. Freiburg: Herder, 1972.

[15] Raspberry Pi Ltd. „Raspberry Pi 5: Specification," besucht am 3. Nov. 2026. Adresse: https://www.raspberrypi.com/products/raspberry-pi-5/

[16] Broadcom. „Spring Boot Reference Documentation," besucht am 3. Nov. 2026. Adresse: https://docs.spring.io/spring-boot/index.html

[17] The Thymeleaf Team. „Thymeleaf Documentation," besucht am 3. Nov. 2026. Adresse: https://www.thymeleaf.org/documentation.html

[18] H2 Group. „H2 Database Engine," besucht am 3. Nov. 2026. Adresse: https://www.h2database.com/html/main.html

[19] *DIN EN ISO 9241-11:2018-11 Ergonomie der Mensch-System-Interaktion – Teil 11: Gebrauchstauglichkeit: Begriffe und Konzepte (ISO 9241-11:2018)*, DIN Deutsches Institut für Normung e. V., Berlin: Beuth, 2018.

[20] J. Nielsen. „10 Usability Heuristics for User Interface Design," besucht am 1. Okt. 2026. Adresse: https://www.nngroup.com/articles/ten-usability-heuristics/

[21] A. Herrmann, *Grundlagen der Anforderungsanalyse: Standardkonformes Requirements Engineering*. Wiesbaden: Springer Vieweg, 2022. DOI: 10.1007/978-3-658-35460-2

[22] N. Döring und J. Bortz, *Forschungsmethoden und Evaluation in den Sozial- und Humanwissenschaften*, 5. Aufl. Berlin, Heidelberg: Springer, 2016. DOI: 10.1007/978-3-642-41089-5

[23] D. Clegg und R. Barker, *Case Method Fast-Track: A RAD Approach*. Wokingham: Addison-Wesley, 1994.

[24] Mozilla Developer Network. „Progressive web apps," besucht am 3. Nov. 2026. Adresse: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps

[25] C. Liebel, *Progressive Web Apps: Das Praxisbuch*. Bonn: Rheinwerk, 2019, ISBN: 978-3-8362-6494-5.

[26] Wanderlog. „Wanderlog Pro: Preisvergleich und Funktionen," besucht am 2. Nov. 2026. Adresse: https://wanderlog.com/pro

[27] TripIt. „TripIt: Travel Planner. App Store Deutschland," besucht am 2. Nov. 2026. Adresse: https://apps.apple.com/de/app/tripit-travel-planner/id311035142

[28] Anki. „AnkiMobile Flashcards. App Store Deutschland," besucht am 2. Nov. 2026. Adresse: https://apps.apple.com/de/app/ankimobile-flashcards/id373493387

[29] LingoDeer. „LingoDeer: Sprachen lernen. App Store Deutschland," besucht am 2. Nov. 2026. Adresse: https://apps.apple.com/de/app/lingodeer-sprachen-lernen/id1261193709

[30] LingoDeer. „LingoDeer Premium: Preise," besucht am 2. Nov. 2026. Adresse: https://www.lingodeer.com/de/pricing

[31] S. Morley. „AdventureLog: Self-hosted travel companion," besucht am 2. Nov. 2026. Adresse: https://adventurelog.app/